CV Architect

Documentazione tecnica

Architettura, tecnologie, infrastruttura di rilascio, repository e procedure operative.

Versione del codice: commit `82adea8` (ramo `main`) — 19 ottobre 2026

Autore: eas80

Indice

1. Panoramica 3
Funzionalità principali 3
2. Stack tecnologico 3
3. Architettura 4
Moduli del codice 4
Integrazione con Claude 5
Crediti e candidature 5
4. Database 5
Sicurezza dei dati 6
5. Pagamenti 6
Cliente Stripe 7
Listino 7
Controlli sull'erogazione 7
6. Repository 7
7. Rilascio e infrastruttura 8
Ambienti 8
Configurazione Render (render.yaml) 8
Avvisi al gestore 9
Variabili d'ambiente 9
8. SEO 10
9. Procedure operative 10
Avvio in locale 10
Deploy 10
Gestione utenti 10
Migrazione dei dati 11
Monitoraggio 11

1. Panoramica

CV Architect è un'applicazione web che, con un percorso guidato, aiuta l'utente a preparare una candidatura: importa il CV (PDF o testo), lo adatta a un annuncio di lavoro, genera la lettera di presentazione e i contenuti del profilo LinkedIn, e produce i PDF finali con template ottimizzati per i sistemi ATS.

È un prodotto commerciale a crediti: **1 credito = 1 candidatura completa** (CV, annuncio e LinkedIn). Il credito si scala una sola volta, alla prima consegna (download, salvataggio o testi LinkedIn). I crediti si acquistano con Stripe e scadono dopo 24 mesi. Gli account amministratore hanno uso illimitato.

Funzionalità principali

- Registrazione con nome, cognome, email e password, conferma dell'indirizzo con codice a 6 cifre via email, accettazione di Termini e Informativa privacy; reset della password via email; accesso ricordato dal browser per 30 giorni.
- Importazione del CV da PDF o testo e conversione in dati strutturati con Claude.
- Estrazione automatica del testo di un annuncio da URL (scraper) o incolla manuale.
- CV adattato all'annuncio con punteggio di compatibilità, lettera di presentazione e revisioni mirate scritte a parole.
- Contenuti LinkedIn (titolo, informazioni, post) in più toni di voce.
- PDF di CV e lettera in italiano o inglese con 4 template ATS-friendly: Classico, Moderno, Tecnico, Executive.
- Archivio «Le mie candidature», attivabile per singolo utente: riapertura e modifica senza nuovi crediti.
- Crediti per candidatura con scadenza a 24 mesi, gestiti per lotti; tetti alle operazioni con l'AI per candidatura e al giorno.
- Acquisto di pacchetti con Stripe Checkout, listino modificabile dal pannello admin, rinuncia al recesso registrata.
- Modulo «Contattaci», Termini e condizioni e Privacy e cookie consultabili nell'app, eliminazione self-service dell'account.
- SEO: pagina iniziale con meta tag, anteprima social, dati strutturati, `robots.txt` e `sitemap.xml`.
- Due ambienti separati: produzione (Stripe live) e prova (Stripe test).
- Avvisi email al gestore per ogni nuova registrazione e ogni acquisto, con l'ambiente nell'oggetto ([PROD] o [TEST]).

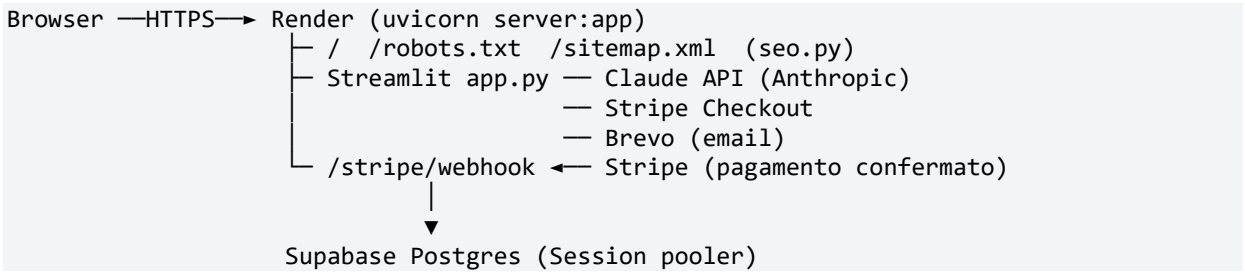
2. Stack tecnologico

Livello	Tecnologia	Versione
Linguaggio	Python	3.13
Interfaccia web	Streamlit	1.65
Server ASGI	Uvicorn (Starlette)	0.54 (Starlette 1.8)
Intelligenza artificiale	Anthropic SDK — modello Claude Sonnet 5.5 (`claude-sonnet-5-5`), Structured Outputs	anthropic 1.13
Validazione dati	Pydantic	2.14
Database	PostgreSQL su Supabase	Postgres 17
Driver database	psycopg 3 + psycopg-pool	3.3

Livello	Tecnologia	Versione
Pagamenti	Stripe (Checkout + webhook)	stripe 16.0
Email transazionali	Brevo (API HTTP)	—
Generazione PDF	ReportLab	5.0
Lettura e anteprima PDF	pypdf, pypdfium2	6.20, 5.14
Scraping annunci	requests + BeautifulSoup	bs4 4.15
Template testi	Jinja2	3.1
Configurazione	python-dotenv (file `.env`)	≥ 1.0

3. Architettura

L'applicazione gira in **un solo processo**: `server.py` crea un'app ASGI (`st.App`) che serve l'interfaccia Streamlit e, sullo stesso indirizzo, le rotte aggiuntive: il webhook Stripe `/stripe/webhook` e le rotte SEO (`/`, `/robots.txt`, `/sitemap.xml`, immagini e icone). Le rotte dell'app hanno la precedenza su quelle di Streamlit. Uvicorn la espone sulla porta assegnata dalla piattaforma.



Moduli del codice

File	Ruolo
`app.py`	Interfaccia Streamlit: percorso guidato, account, menu, crediti e acquisti, contatti, pagine legali, pannello admin.
`server.py`	Avvio in produzione: Streamlit, webhook e rotte SEO nello stesso processo ASGI.
`seo.py`	Pagina iniziale con meta tag, Open Graph, dati strutturati schema.org e testo statico; `robots.txt`, `sitemap.xml`, icone.
`webhook_server.py`	Rotta `/stripe/webhook`: verifica la firma e accredita i crediti. Avviabile anche da sola in locale (porta 8521).
`editors.py`	Editor manuali di CV, lettera e testi LinkedIn.
`config.py`	Configurazione centralizzata letta dalle variabili d'ambiente.
`schemas/cv_schema.py`	Modelli Pydantic: CV, risultato del match, contenuti LinkedIn.
`services/claude_service.py`	Chiamate a Claude: parsing del CV, tailoring, revisioni, LinkedIn.
`services/db.py`	Pool di connessioni Postgres, schema delle tabelle, RLS e ruolo applicativo.
`services/auth_service.py`	Utenti, password, codici email, sessioni di accesso, crediti per lotti, impostazioni, statistiche.

File	Ruolo
`services/payment_service.py`	Listino dei pacchetti, sessioni Stripe Checkout, erogazione dei crediti.
`services/email_service.py`	Email con Brevo: codici di conferma e reset, messaggi del modulo «Contattaci», avvisi al gestore.
`services/applications_service.py`	Archivio candidature isolato per utente.
`services/pdf_service.py`	Rendering PDF di CV e lettera con i template ATS.
`services/scraper_service.py`	Estrazione del testo di un annuncio da una pagina web.
`legal/*.md`	Termini e condizioni, Informativa privacy e cookie (mostrati nell'app con `?doc=termini` / `?doc=privacy`).
`assets/`	Immagine di anteprima social (1200×630) e icone del sito.
`manage_users.py`	Gestione utenti da riga di comando.
`migrate_sqlite_to_postgres.py`	Migrazione una tantum dal vecchio `database.db` SQLite a Postgres.
`run_server.pyw`	Avvio in background su Windows (Utilità di pianificazione), log in `logs/`.

Integrazione con Claude

Tutte le chiamate usano gli **Structured Outputs** dell'SDK Anthropic: lo schema Pydantic viene imposto alla risposta, che torna già validata. Modello, limite di token, livello di effort e timeout sono configurabili (`CLAUDE_MODEL`, `CLAUDE_MAX_TOKENS`, `CLAUDE_EFFORT`, `CLAUDE_TIMEOUT_SECONDS`). Ogni operazione riuscita viene registrata in `usage_log`.

Durante l'elaborazione l'interfaccia mostra un riquadro di stato e il pulsante diventa « ⏳ …» disabilitato, così l'utente vede che l'app sta lavorando e non può ripetere il clic.

Crediti e candidature

- Una «candidatura» è la sessione di lavoro dal CV letto fino a «Nuova candidatura» (o a un nuovo CV caricato). Le operazioni con l'AI al suo interno non costano singolarmente.
- Il credito si scala alla **prima consegna**: download di un documento, salvataggio nell'archivio o generazione dei testi LinkedIn. Una candidatura fatta tutta a mano, senza AI, non costa nulla.
- Senza crediti le funzioni AI sono bloccate in partenza; se il saldo scende a zero a metà candidatura, la consegna viene bloccata con l'invito all'acquisto.
- Tetti per contenere i costi: `SESSION_AI_CALLS_LIMIT` operazioni per candidatura (default 20) e `DAILY_AI_CALLS_LIMIT` per utente nelle ultime 24 ore (default 40). Gli admin non hanno limiti.
- Riaprire una candidatura salvata non consuma crediti: risulta già pagata.
- **Scadenza:** ogni lotto di crediti (acquisto, regalo dell'admin, crediti iniziali) scade dopo 24 mesi. Si consumano prima i lotti che scadono prima (FIFO); i lotti scaduti vengono azzerati alla lettura dell'utente. Il saldo `users.credits` resta la somma dei crediti residui dei lotti.

4. Database

PostgreSQL 17 ospitato su **Supabase**, con un progetto per ambiente (vedi capitolo 7). L'app si collega tramite il **Session pooler** (porta 5432), raggiungibile anche da server senza IPv6 come Render. Lo schema è

creato e aggiornato automaticamente a ogni avvio (`ensure_schema`, istruzioni idempotenti), protetto da un advisory lock contro avvii concorrenti.

Tabella	Contenuto
`users`	Email, nome e cognome, hash della password, saldo crediti (mai negativo), flag admin, email confermata, data e versione dei Termini accettati, abilitazione all'archivio, id del cliente Stripe per modalità (`stripe_customer_test`, `stripe_customer_live`).
`email_codes`	Codici a 6 cifre per conferma email e reset password: solo l'hash, scadenza (15 minuti) e tentativi (massimo 5).
`credit_lots`	Lotti di crediti: origine (`purchase`, `admin`, `initial`, `legacy`), sessione Stripe, crediti iniziali e residui, crediti scaduti, scadenza.
`payments`	Acquisti erogati; chiave primaria = id della sessione Stripe (erogazione idempotente); `livemode` distingue pagamenti veri e di prova. Restano anche se l'account viene eliminato.
`login_sessions`	Accessi ricordati dal browser: solo l'hash del token e la scadenza.
`settings`	Impostazioni del pannello admin: uso gratuito senza AI, listino dei pacchetti (`credit_packs`, JSON).
`usage_log`	Una riga per ogni operazione con Claude andata a buon fine (resta, senza utente, se l'account viene eliminato).
`saved_applications`	Candidature salvate: CV e match in JSONB, lettera, contenuti LinkedIn, punteggio, dati per la riapertura.

All'avvio, i saldi non coperti da lotti (account precedenti alla scadenza dei crediti) diventano un lotto `legacy` con scadenza a 24 mesi. L'eliminazione di un account cancella in cascata codici, sessioni, lotti e candidature; pagamenti e statistiche restano con `user_id` NULL.

Sicurezza dei dati

- **Row Level Security** attiva su tutte le tabelle, senza policy per i ruoli `anon` e `authenticated`: la Data API pubblica di Supabase non può leggere né scrivere.
- Le candidature sono lette e scritte con il ruolo `cv_app_user` (NOLOGIN, NOBYPASSRLS), soggetto alla policy `user_id = app.current_user_id`: anche una query errata non vede i dati di altri utenti.
- Password e codici email salvati solo come hash **PBKDF2-SHA256** (600.000 iterazioni, sale casuale); password di almeno 8 caratteri.
- I token di sessione nel cookie sono salvati nel database solo come hash.
- Pool di connessioni con verifica prima del riuso (Supabase chiude le connessioni inattive) e prepared statement disattivati, compatibili anche con il pooler in transaction mode.
- Statistiche di utilizzo di Streamlit disattivate (`gatherUsageStats = false`).

5. Pagamenti

Gli acquisti usano **Stripe Checkout** in euro. L'app crea la sessione di pagamento; i crediti vengono erogati dal webhook dopo che Stripe ha confermato il pagamento (e, come rete di sicurezza, da un controllo diretto della sessione mentre la scheda è aperta: l'erogazione è idempotente). La firma di ogni evento è verificata con `STRIPE_WEBHOOK_SECRET`.

Cliente Stripe

Al primo acquisto l'app crea un **cliente Stripe** con nome, cognome ed email dell'utente (lingua italiana, `user_id` nei metadati) e ne salva l'id in `users`, separato per modalità test e live; gli acquisti successivi lo riusano. Se il cliente è stato eliminato dalla dashboard di Stripe, la sessione fallisce con `resource_missing` e l'app ne crea uno nuovo. Il nome è anche nei metadati della sessione (`customer_name`). Gli account senza nome lo inseriscono nella finestra «Crediti» prima di pagare.

Il pulsante «Procedi al pagamento» si disattiva e cambia testo durante la creazione della sessione, poi viene sostituito dal link «Paga con Stripe»: niente doppi clic.

Listino

Il listino si imposta dal pannello admin («Pacchetti e prezzi») ed è salvato nella tabella `settings`. Regole: da 1 a 6 pacchetti, candidature da 1 a 1000 e tutte diverse, prezzo da 0,50 € (minimo Stripe) a 1.000 €. Il listino è riletto dal database al più ogni 30 secondi. Senza impostazioni vale il listino predefinito:

Pacchetto	Crediti	Prezzo	A candidatura
1 candidatura	1	3,99 €	3,99 €
3 candidature	3	8,99 €	2,99 €
10 candidature	10	19,99 €	1,99 €

Il prezzo «a candidatura» è arrotondato per difetto; il più basso del listino è quello mostrato in home page («da 1,99 € a candidatura»).

Controlli sull'erogazione

- Candidature e prezzo del pacchetto sono scritti nei metadati della sessione (`credits`, `amount_cents`): un cambio di listino non blocca i pagamenti già avviati. Per le sessioni precedenti si usa il listino attuale.
- I crediti si erogano solo se l'importo pagato (`amount_total`) e la valuta corrispondono.
- Una sessione dell'altra modalità (live con chiavi test o viceversa) viene ignorata e scritta nel log.
- Prima del pagamento l'utente deve spuntare la rinuncia al recesso di 14 giorni (contenuti digitali, art. 59 lett. o Codice del Consumo); data e ora restano nei metadati (`withdrawal_waiver_at`).

Eventi gestiti dal webhook: `checkout.session.completed` e `checkout.session.async_payment_succeeded`. Endpoint: `https://<servizio>.onrender.com/stripe/webhook`.

6. Repository

Voce	Valore
Hosting	GitHub
URL	https://github.com/eas80/cv-archtect
Ramo `main`	Produzione: deploy automatico su `cv-architect` a ogni push.
Ramo `staging`	Prova: deploy automatico su `cv-architect-test`.
Ultimo commit	`82adea8` — Avviso email all'amministratore per nuove registrazioni e acquisti

Flusso di rilascio: le modifiche si pubblicano prima su `staging`, si provano sull'ambiente di prova e, dopo l'approvazione, si portano su `main`.

Esclusi dal repository tramite `.gitignore`: `.env` e ogni `.env.*` tranne `.env.example` (chiavi e password), `database.db`, `logs/`, ambienti virtuali e cache Python.

7. Rilascio e infrastruttura

Componente	Servizio	Dettagli
Applicazione web	Render (Web Service)	Due servizi definiti in `render.yaml` (Blueprint), regione Frankfurt, piano Free.
Database	Supabase	Postgres 17, un progetto per ambiente, Session pooler.
Intelligenza artificiale	Anthropic API	Modello Claude Sonnet 5.5.
Pagamenti	Stripe	Checkout e webhook; modalità test in prova, live in produzione.
Email	Brevo	API HTTP (Render Free blocca le porte SMTP); 300 email al giorno nel piano gratuito.
Codice sorgente	GitHub	eas80/cv-archtect

Ambienti

	Produzione	Prova
Servizio Render	`cv-architect`	`cv-architect-test`
Ramo git	`main`	`staging`
Indirizzo	https://cv-architect.onrender.com	https://cv-architect-test.onrender.com
Stripe	chiavi live (`sk_live_...`)	chiavi test (`sk_test_...`)
Supabase	progetto `hucvzfexcfyttklpmczl`, eu-west-1 (Irlanda)	progetto `qftahawmxokddqmqdirr`, eu-west-2 (Londra)
Motori di ricerca	indicizzato	escluso (`noindex`, `robots.txt` Disallow)

Con chiavi Stripe di test l'app mostra in alto la fascia «🧪 Modalità test»; il pannello admin indica la modalità attiva e conta vendite e incasso solo di quella modalità.

Configurazione Render (render.yaml)

```
runtime:          python (3.13, da .python-version)
region:           frankfurt
branch:           main (cv-architect) / staging (cv-architect-test)
buildCommand:     pip install -r requirements.txt
startCommand:     uvicorn server:app --host 0.0.0.0 --port $PORT
healthCheckPath:  /_stcore/health
autoDeploy:       true
```

Render non ha una regione in Irlanda: Frankfurt è la più vicina al database. Con il piano Free il servizio si spegne dopo 15 minuti senza visite e impiega circa un minuto a ripartire; per l'uso commerciale è consigliato il piano Starter.

Avvisi al gestore

`email_service.notify_admin` invia a `NOTIFY_EMAIL` un'email per ogni nuova registrazione (da `auth_service.register`) e ogni acquisto erogato (da `payment_service.fulfill_checkout`, quindi una sola volta per sessione). L'oggetto inizia con l'ambiente: `[PROD]` sul ramo `main`, `[TEST]` su `staging`, `[LOCALE]` altrove (da `RENDER_GIT_BRANCH`, forzabile con `APP_ENV`). L'invio avviene in un thread in background e gli errori vanno solo nel log.

Variabili d'ambiente

Variabile	Obbligatoria	Descrizione
`DATABASE_URL`	Sì	Stringa di connessione del Session pooler Supabase.
`ANTHROPIC_API_KEY`	Sì	Chiave dell'API Anthropic.
`STRIPE_SECRET_KEY`	Per i pagamenti	`sk_test_...` in prova, `sk_live_...` in produzione: determina la modalità.
`STRIPE_WEBHOOK_SECRET`	Per i pagamenti	Segreto `whsec_...` dell'endpoint webhook della stessa modalità.
`BREVO_API_KEY`, `EMAIL_SENDER`	Per le email	Chiave Brevo e mittente verificato. Senza chiave le email finiscono nel log.
`CONTACT_EMAIL`	No	Destinatario del modulo «Contattaci» (default cv.architect.info@gmail.com).
`NOTIFY_EMAIL`	No	Destinatario degli avvisi su registrazioni e acquisti (default `CONTACT_EMAIL`; vuota = nessun avviso).
`APP_ENV`	No	Nome dell'ambiente negli avvisi; di default PROD su `main`, TEST su `staging`.
`ADMIN_EMAILS`	No	Email degli amministratori, separate da virgola.
`APP_BASE_URL`	No	Indirizzo pubblico; su Render si usa `RENDER_EXTERNAL_URL`. Da impostare con un dominio proprio.
`GOOGLE_SITE_VERIFICATION`	No	Codice del meta tag di verifica di Google Search Console.
`SEO_NOINDEX`	No	`1` esclude il sito dai motori di ricerca (automatico sul ramo `staging`).
`SESSION_AI_CALLS_LIMIT`, `DAILY_AI_CALLS_LIMIT`	No	Tetti alle operazioni con l'AI (default 20 e 40).
`INITIAL_CREDITS`	No	Crediti regalati ai nuovi account (default 0).
`SESSION_DAYS`	No	Durata dell'accesso ricordato (default 30).
`APP_TIMEZONE`	No	Fuso orario delle date (default Europe/Rome).
`CLAUDE_MODEL`, `CLAUDE_EFFORT`	No	Modello e livello di effort di Claude.
`DB_POOL_MAX_SIZE`	No	Connessioni massime del pool (default 5).

I valori segreti non stanno nel repository: in locale si trovano in `.env`, su Render si inseriscono nella dashboard (`sync: false` in `render.yaml`).

8. SEO

Streamlit costruisce la pagina via JavaScript e il suo `index.html` è vuoto. `seo.py` serve la rotta `/` con l'`index.html` di Streamlit arricchito:

- `lang="it"`, titolo, meta description e keywords, `canonical`, `theme-color`, `robots`;
- Open Graph e Twitter Card con l'immagine `/og-image.png` (anteprima nei link condivisi);
- dati strutturati JSON-LD: `WebApplication` con `AggregateOffer` (prezzi dal listino) e `FAQPage`;
- dentro `#root` un testo statico (titolo, come funziona, prezzi, domande frequenti, link legali), sostituito dall'app al caricamento;
- pagine `?doc=termini` e `?doc=privacy` con titolo e descrizione propri;
- icone proprie al posto di quella di Streamlit (`/favicon.png`, `/icon-180.png`, `/icon-512.png`).

Prezzi e domande frequenti seguono il listino attuale; la pagina è rigenerata quando il listino cambia. `/robots.txt` esclude `/_stcore/` e `/stripe/` e indica la sitemap; `/sitemap.xml` elenca home e pagine legali.

9. Procedure operative

Avvio in locale

```
python -m venv venv
venv\Scripts\pip install -r requirements.txt
copy .env.example .env      (poi compilare le variabili)
venv\Scripts\python -m uvicorn server:app --port 8520
```

L'app è disponibile su http://localhost:8520. Per ricevere i webhook in locale: `stripe listen --forward-to localhost:8520/stripe/webhook`.

Deploy

- Primo rilascio: su Render **New → Blueprint**, collegare il repository e compilare le variabili dei due servizi.
- Rilasci successivi: push su `staging` per la prova, poi su `main` per la produzione (deploy automatico).
- Webhook Stripe: Developers → Webhooks → Add endpoint nella stessa modalità della chiave, poi copiare il signing secret in `STRIPE_WEBHOOK_SECRET`.
- Passaggio a Stripe live: attivare l'account Stripe, impostare `sk_live_...` e il `whsec_...` live su `cv-architect`, verificare nel pannello admin «Stripe: modalità live».
- Google Search Console: proprietà «Prefisso URL», verifica con `GOOGLE_SITE_VERIFICATION`, invio di `sitemap.xml`.

Gestione utenti

Dal pannello admin: saldo crediti, abilitazione all'archivio, listino, uso gratuito senza AI, statistiche. Da riga di comando:

```
python manage_users.py list
python manage_users.py credits utente@example.com 10
python manage_users.py admin utente@example.com on
python manage_users.py verified utente@example.com on
python manage_users.py password utente@example.com NuovaPassword
python manage_users.py delete utente@example.com
```

Migrazione dei dati

`migrate_sqlite_to_postgres.py` copia utenti, pagamenti, impostazioni, utilizzo e candidature dal vecchio `database.db` a Postgres e riallinea i contatori degli id. Funziona solo su un database vuoto. La migrazione è stata eseguita il 16 ottobre 2026 (5 utenti, 1 pagamento, 6 voci di utilizzo, 1 candidatura).

Monitoraggio

- Health check: `GET /_stcore/health` (usato da Render).
- Log applicativi: dashboard di Render (produzione e prova) o `logs/streamlit.log` (avvio Windows in background).
- Pagamenti ed eventi webhook: dashboard Stripe → Developers → Webhooks (separati per modalità test e live).
- Email inviate: dashboard Brevo → Transactional.